SYSTEM ARCHITECTURE · CA FIRM SAAS

How this project actually works

Boot order, the request lifecycle end to end, every layer's job, and what each folder does — for both the Express server and the React client.

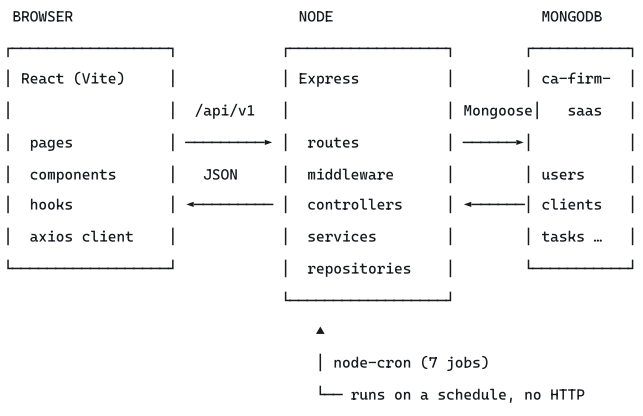
168 server files **98** client components **21** mounted route groups **18** backend features **7** cron jobs

1 · The big picture

TWO APPLICATIONS, ONE REPOSITORY

This is an **npm workspaces monorepo**: `client` and `server` are separate packages that ship together. They are genuinely separate programs — different module systems, different runtimes, talking only over HTTP/JSON.

	SERVER	CLIENT
Runtime	Node.js	The browser
Modules	CommonJS (`require`)	ES Modules (`import`)
Entry	`server/src/server.js`	`client/index.html` → `main.jsx`
Framework	Express	React 19
Dev port	5000	5173 (Vite)
Talks to	MongoDB via Mongoose	The API via axios



In development the two ports look like one. Vite proxies anything starting with `/api` to `localhost:5000` (configured in `client/vite.config.js`). So the browser only ever sees `localhost:5173` — no CORS involved. In production the Express server serves the built React files itself, so it's genuinely one origin.

2 · Server boot sequence

WHAT RUNS, IN WHAT ORDER, WHEN YOU TYPE NPM RUN DEV

`npm run dev` uses `concurrently` to launch two processes. The server one runs `nodemon src/server.js`. Here is exactly what happens:

1 **`server/src/server.js — line 1`**
`require('./models/registerModels')`. This is first *on purpose*: every Mongoose model registers itself as a side effect of being imported, and `.populate()` fails if a referenced model isn't registered yet. This file imports all ~20 models up front.

2 **`config/env.js`**
Loads `.env` via dotenv and builds one frozen config object. It also calls `assertProductionConfig()` — in production, missing `MONGO_URI` or JWT secrets crash the process immediately rather than failing mysteriously later.

3 **`config/db.js → connectDB()`**
Attaches connection event loggers, then `await mongoose.connect(env.mongoUri)`. Nothing else starts until the database is reachable.

4 **`app.js`**
Builds the Express app: security headers, CORS, body parsers, rate limiting, then mounts `/api/v1`. (Detailed in the next section.)

5 **`app.listen(env.port)`**
The server starts accepting requests on port 5000.

6 **`jobs/index.js → registerJobs()`**
Registers 7 cron schedules — skipped when `NODE_ENV=test` so the test suite doesn't fire background work.

7 **`SIGINT / SIGTERM handlers`**
Graceful shutdown: stop accepting connections, finish in-flight requests, then exit.

> **Why `app.js` and `server.js` are separate files.** `app.js` exports a configured Express app but never calls `listen()`. That lets the Jest tests import the app and fire requests at it in-memory, with no real port and no real database (they use `mongodb-memory-server`). Separating "build the app" from "start the app" is what makes the suite fast.

---

3 · The middleware pipeline

APP.JS — ORDER IS BEHAVIOUR, NOT STYLE

Express runs middleware **top to bottom**. Each one either responds, or calls `next()` to pass control on. This is the exact order in `app.js`:

#	MIDDLEWARE	WHY IT SITS THERE
1	`helmet()`	Security headers first, so they apply to every response including errors. CSP disabled because the server also serves the React app.
2	`cors()`	Must run before routes so preflight `OPTIONS` is answered. Allows origins from `env.corsOrigins`; a missing `Origin` header (same-origin, curl, proxy) is always allowed.
3	`express.json()`	Parses JSON bodies into `req.body`. Capped at 2 MB.
4	`express.urlencoded()`	Parses form-encoded bodies.
5	`cookieParser()`	Populates `req.cookies` — the refresh token lives in an httpOnly cookie, so this must precede auth.
6	`mongoSanitize()`	Strips `$` and `.` keys so a crafted body can't inject Mongo operators. Runs *after* parsing (nothing to sanitise before) and *before* routes.

7	`apiLimiter`	Global rate limit. Auth routes add a second, stricter limiter of their own.
8	`morgan`	Request logging, piped into the app's logger.
9	`/api/v1/health`	Liveness probe — deliberately above the auth-protected router.
10	`/api/v1` → routes	The real application. All 21 feature routers.
11	static + SPA fallback	*Production only.* Serves `client/dist` ; any non-`/api` GET returns `index.html` so client-side routes survive a refresh.
12	`notFoundHandler`	Nothing matched → produce a 404 `AppError` .
13	`globalErrorHandler`	Last. Has 4 arguments, which is how Express recognises an error handler.

How errors reach the end

Controllers are wrapped in `asyncHandler` , which catches any rejected promise and forwards it to `next(err)` . That means **no controller in this codebase contains try/catch**. Everything lands in `globalErrorHandler` , which normalises it:

THROWN	BECOMES
`AppError` (thrown deliberately)	Its own status + code
Mongoose `CastError`	400 — "Invalid id"
Mongoose `ValidationError`	422 with the field list
Mongo duplicate key (11000)	409 — "x already exists"
`JsonWebTokenError` / expired	401 — "Invalid or expired session"
Anything else	500, stack logged server-side, generic message to the client

Every response — success or failure — has the same envelope, which is what lets the frontend handle all of them uniformly:

```
{ success: true,  message: 'Success', data: {...}, meta?: {...} }
{ success: false, message: 'Client not found', data: null,
  error: { code: 'NOT_FOUND', details: [...] } }
```

---

4 · Backend layers

EVERY FEATURE FOLDER HAS THE SAME FIVE FILES

All 18 backend features follow one shape. Learn it once and the whole server is readable:

ROUTES

URL → handler, plus the gates

Maps method+path to a controller and attaches `requireAuth` , `authorize('client:read')` , `validate(...)` , file upload. Contains zero logic.

CONTROLLER

Translate HTTP ⇄ service

Reads `req.user` / `req.body` / `req.params` , calls one service function, wraps the result in `ApiResponse` . Never touches the database.

SERVICE

The business rules

Permission checks beyond the route gate, cross-feature coordination, notifications, derived values. Knows nothing about HTTP — no `req` , no `res` .

REPOSITORY **Database access**

Extends `BaseRepository`, which is the *only* place `firmId` scoping and `isDeleted` filtering happen. Most repositories are 8 lines.

MODEL **Shape & field validation**

Mongoose schema, enums, defaults, indexes, plus `auditPlugin` for the shared tenant fields.

Two supporting files appear in most features: `validators/` (plain functions returning `{ isValid, errors, value }` — no Joi/Zod) and `*.model.js` constants such as status enums.

> **The rule that keeps it safe:** controllers and services never build raw Mongo queries. They call repository methods, so tenant isolation and soft-delete can't be forgotten in one feature and enforced in another. If you ever write `Model.find(...)` outside a repository, you've stepped outside the guarantee.

---

5 · A request, end to end

FOLLOWING "OPEN THE CLIENTS PAGE" THROUGH ALL 14 STEPS

```
BROWSER                                              SERVER
──────────────────────────────────────────────────────────────────────────

 1  ClientsPage mounts
 2  useApiData runs its fetcher
 3  clientApi.list({page:1})
 4  axios interceptor adds
    Authorization: Bearer
 5  GET /api/v1/clients?page=1   ──────►  6  helmet → cors → json →
                                             cookies → sanitize → limiter
                                          7  routes/index.js → clientRoutes
                                          8  requireAuth verifies the JWT,
                                             sets req.user {id, role, firmId}
                                          9  authorize('client:read')
                                             checks the capability
                                         10  controller → service
                                             (employee? scope to assignedTo)
                                         11  repository.paginate(firmId,…)
                                             → adds firmId + isDeleted:false
                                         12  MongoDB returns documents
                                         13  ApiResponse.paginated wraps them
14  axios resolves          ◄──────────────   { success, data, meta }
15  useApiData setData → React re-renders → DataTable paints rows
```

What happens when the token has expired

Step 6 returns **401**. The axios response interceptor in `apiClient.js` then, without the page knowing:

1. Marks the request as retried (so it can never loop)
2. Calls `POST /auth/refresh` — the browser automatically sends the httpOnly refresh cookie
3. Stores the new access token in memory and replays the original request
4. If the refresh also fails, it calls the unauthorized handler → `AuthContext` clears state → you land on the login screen

Concurrent 401s share a single in-flight refresh promise, so ten failing requests trigger one refresh, not ten.

6 · Client boot sequence

FROM INDEX.HTML TO A PAINTED PAGE

1. **`client/index.html`**
   A near-empty shell: `<div id="root"></div>` plus `<script type="module" src="/src/main.jsx">`. Vite serves it.

2. **`src/main.jsx`**
   The real entry. Calls `ReactDOM.createRoot(...).render(...)` and — importantly — establishes the provider tree.

3. **`Provider tree`**
   `StrictMode → BrowserRouter → ToastProvider → AuthProvider → BrandingProvider → App`. Order matters: `AuthProvider` is inside the router (it navigates) and inside `ToastProvider` (it raises toasts); `BrandingProvider` is innermost because it needs the logged-in user's firm.

4. **`AuthProvider's boot effect`**
   The session-restore decision, before anything renders — see below.

5. **`App.jsx`**
   Declares every route. Pages are `lazy()`-loaded, so each screen is a separate JS chunk fetched on first visit.

6. **`Route guards`**
   `ProtectedRoute` / `ServiceRoute` / `AdminRoute` / `PortalRoute` decide whether you see the page or get redirected.

7. **`Layout → page`**
   `DashboardLayout` (or `PortalLayout`) renders the chrome; `<Outlet/>` is where the matched page appears.

The session-restore rule

On every page load `AuthProvider` asks two questions before attempting a refresh:

```
if (!hasSessionMarker() || msSinceLastActivity() > IDLE_TIMEOUT_MS) {
  // brand-new tab, or idle too long → go to login
}
```

The marker lives in **sessionStorage**, which survives F5 but dies with the tab. That's what produces the intended behaviour: *refresh keeps you in, closing the tab logs you out* — even though the refresh cookie may still be valid.

7 · Client routing & guards

APP.JSX — ONE FILE, THE WHOLE MAP

```
/                     → RootRedirect  (client → /portal, staff → /services)

GuestRoute            (redirects away if already logged in)
 └ AuthLayout
    /login  /forgot-password  /reset-password

ProtectedRoute        (must be logged in)
 ├ /services                      ServiceLauncherPage
 │
 ├ PortalRoute        (client role only)
 │  └ PortalLayout
 │     /portal  /portal/compliance  /portal/invoices
 │     /portal/ledger  /portal/documents  /portal/tasks  /portal/messages
 │
 └ DashboardLayout
    ├ ServiceRoute "crm"      → /crm/*        dashboard, leads, clients,
    │                                          tasks, reminders, compliance…
    ├ ServiceRoute "khata"    → /khata/*      ledgers, books
    ├ ServiceRoute "accounts" → /accounts/*   invoices, expenses, documents
    │   └ AdminRoute          → team, departments, settings, activity
    └ cross-service           → /users/:id  /notifications  /settings/profile
```

GUARD	QUESTION IT ASKS	IF NO
`ProtectedRoute`	Is anyone logged in?	→ `/login`
`GuestRoute`	Is nobody logged in?	→ their home page
`ServiceRoute`	Does this user have this workspace?	→ `/services`
`AdminRoute`	admin or superadmin?	→ their home page
`PortalRoute`	Is the role `client`?	→ `/services`

> **Guards are convenience, not security.** They stop a user landing on a page that would only error. The real enforcement is `authorize()` and `requireAdminRole` on the server. Deleting every guard would change the UX and change nothing about what data anyone can reach.

How a client page fetches data

Three pieces, used by nearly every screen:

1. **`features/x/services/x.service.js`** — thin axios wrapper, one function per endpoint
2. **`hooks/useApiData.js`** — calls it, tracks `{ data, loading, error, refetch }`, and ignores stale responses via a request-id counter so fast filter changes can't render older results
3. **The page** — renders `loading` / `error` / data, and calls `refetch()` after any mutation

---

8 · Auth & token flow

TWO TOKENS, TWO VERY DIFFERENT JOBS

	ACCESS TOKEN	REFRESH TOKEN
Format	JWT	Opaque random string
Lives in	JavaScript memory only	httpOnly cookie
Lifetime	15 minutes	7 days
Sent as	`Authorization: Bearer …`	Automatically, by the browser
Stored server-side	No — stateless	Yes, as a bcrypt hash

Readable by JS?	Yes (that's why it's short-lived)	No — XSS can't steal it

The access token carries only `{ userId, role, firmId, sessionId }`. `requireAuth` verifies the signature and trusts those claims *without hitting the database* — that's the point of a stateless token. Routes that need live data (e.g. the Client Portal checking a freshly-changed link) use their own middleware to load the user.

Login

1. `POST /auth/login` → is the account locked? (5 failures = 15 min)
2. `bcrypt.compare` the password
3. 2FA enabled? → return a 10-minute `tempToken`, *not* a session. Only `/2fa/verify-login` can redeem it
4. Otherwise create a `RefreshToken` row, sign the access token with its id as `sessionId`, set the cookie

Rotation & theft detection

Each refresh revokes the old token and issues a new one, linked by `replacedBy`. If an *already-revoked* token is presented, that means someone replayed a stolen one — so the server revokes **every** session for that user, not just the offending one.

---

9 · Multi-tenancy & permissions

THREE INDEPENDENT GATES

Gate 1 — firm isolation

Every tenant-owned document carries `firmId`, added by `auditPlugin`. `BaseRepository.scoped()` injects `{ firmId, isDeleted: false }` into every query. Firms are therefore invisible to each other structurally, not by convention.

Gate 2 — capabilities

`rbac.middleware.js` maps a role to a capability list, and routes ask for a capability, never a role name:

```
router.delete('/:id', authorize('client:delete'), controller.remove);
```

ROLE	CAPABILITIES
`superadmin`	`['*']` — everything
`admin`	`['*']` — everything (differs from superadmin in *who it can manage*, not what it can use)
`employee`	Full CRUD minus deletes, settings and user management
`client`	`['user:read']` only — the Portal is their sole data path

A separate pair of helpers, `canManageTargetRole()` and `assignableRoles()`, answers "who may edit whom": superadmin manages anyone; admin manages only employees and clients.

Gate 3 — row scoping

An employee holds `client:read`, but should only see *their* clients. So `client.service` and `lead.service` take an optional `actor` argument: when the role is employee, lists are filtered to `assignedTo === actor.id`, and fetching an unassigned record returns **404, not 403** — deliberately, so the response never confirms the record exists.

> **Why the three gates are separate.** They fail independently. Capability bugs can't break firm isolation; scoping bugs can't grant delete rights. Each is enforced in exactly one place — `BaseRepository`, `rbac.middleware`, and the two services respectively.

---

10 · Background jobs

JOBS/INDEX.JS — THE PART THAT RUNS WITH NOBODY LOGGED IN

TIME	JOB	DOES
01:00	`flagOverdueInvoices`	Marks invoices past their due date
02:00 Sun	`cleanupRefreshTokens`	Purges long-revoked tokens
05:30	`complianceGenerate`	Creates the current period's returns for every client (idempotent upsert)
06:00	`generateRecurringReminders`	Rolls recurring reminders forward
07:00	`reminderDigest`	Notifies assignees of today's reminders
08:00	`complianceReminders`	Emails clients about pending returns inside their window
09:30	`invoiceFollowUps`	One payment nudge, 10 days after issue

Each is wrapped in `runSafely()`, so a thrown error is logged and the scheduler keeps running — one broken job can never take down the others or the server. Several check `isAutomationEnabled(firmId, key)` first, which is what makes the Automation Rules toggles in Settings genuinely functional rather than cosmetic.

---

11 · Folder map

WHERE TO LOOK FOR WHAT

server/src

```
server.js              entry — boot sequence
app.js                 express app + middleware order
config/                env.js (all config), db.js, cloudinary, mailer
routes/index.js        mounts all 21 feature routers under /api/v1
middlewares/           auth, rbac, validate, errorHandler, rateLimiter, upload
models/                registerModels.js, ActivityLog, plugins/auditPlugin
repositories/          BaseRepository — firm scoping + soft delete live here
utils/                 ApiResponse, AppError, asyncHandler, pagination, jwt,
                       twoFactor, logger, setPassword (CLI), seed
jobs/                  7 cron jobs + index.js
features/<name>/       routes · controllers · services · repositories ·
                       models · validators   ← the repeating unit
```

client/src

```
main.jsx               entry — provider tree
App.jsx                every route
index.css              design tokens (--color-*), light/dark themes
services/apiClient.js  axios instance + auth interceptors
constants/             roles, services (sidebar menus), session,
                       assistantKnowledge, assistantActions
hooks/                 useApiData, useTheme, useCreateParam
layouts/               DashboardLayout (staff) · PortalLayout (client)
components/ui/         Button, Input, Card, Modal, DataTable, Badge…
components/layout/     Sidebar, Topbar
components/shared/     ProtectedRoute, Toast, GlobalSearch, PanelAssistant…
features/<name>/       pages · components · services   ← the repeating unit
```

Where to start for a given task

YOU WANT TO…	OPEN
Add an API endpoint	`server/src/features/<x>/routes/`, then work down the layers
Add a page	`client/src/features/<x>/pages/` + register in `App.jsx`
Add a sidebar link	`client/src/constants/services.js`
Change who can do what	`server/src/middlewares/rbac.middleware.js` (+ mirror in `constants/roles.js`)
Change colours / spacing	`client/src/index.css` custom properties

Change a scheduled behaviour	`server/src/jobs/`
Understand a past decision	`PROJECT_STATE.md`

CA Firm SaaS — architecture reference. Companions: `README.md` (setup) · `PROJECT_STATE.md` (feature-by-feature history and rationale).